



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 01

NOMBRE COMPLETO: Cordero Miranda Evelyn Patricia

N° de Cuenta: 317314975

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 19 de febrero del 2026

CALIFICACIÓN: _____

EJERCICIOS

1.- Cambio de color de fondo

Como primera actividad se nos pidió cambiar el color de fondo de la ventana de manera aleatoria, tomando el rango de colores RGB con una periodicidad de 2 segundos.

Al inicio del código se incluyeron las librerías `ctime` para el control del tiempo y `cstdlib` para las funciones con las que ya se ha trabajado anteriormente como `srand` y `rand`, además de declarar variables para controlar el tiempo inicializándola en 0 y variables para nuestro código de colores RGB inicializándolas en 1 (dando por resultado el color blanco)

```
8      // CAMBIO 1: -----
9      #include <ctime>    // Para time()
10     #include <cstdlib>  // Para rand() y srand()
11     // Variables para el control del tiempo y color
12     float tiempoAnterior = 0.0f;
13     float r = 1.0f, g = 1.0f, b = 1.0f; // Color inicial (Blanco)
14     // -----
```

Dentro del main principal usamos la función `srand` para generar números aleatorios dependientes del tiempo, garantizando así aleatoriedad con cada ejecución única, debido a que siempre tendremos una hora irrepetible de ejecución.

```
155     ~ int main()
156     {
157     ~     // CAMBIO 2: -----
158         // Inicializar semilla aleatoria
159         srand(static_cast<unsigned int>(time(0)));
160         //-----
```

Finalmente, dentro de nuestro `while (!glfwWindowShouldClose(mainWindow))` que se repite cada ciclo de reloj, obtenemos el tiempo actual a partir del tiempo transcurrido desde que iniciamos GLFW, si este tiempo actual menos el tiempo anterior es igual o mayor a 2 segundos, generamos números aleatorios para las variables `r`, `g` y `b`, mismas que se usarán en la función `glClearColor`

```
207     ~ while (!glfwWindowShouldClose(mainWindow))
208     {
209         //Recibir eventos del usuario
210         glfwPollEvents();
211     ~     // Cambio 3: -----
212         // Control del tiempo para el cambio de color (cada 2.0 segundos)
213         float tiempoActual = (float)glfwGetTime();
214     ~         if (tiempoActual - tiempoAnterior >= 2.0f)
215         {
216             // Generar numeros para el RGB de forma aleatoria
217             r = static_cast<float>(rand()) / static_cast<float>(RAND_MAX);
218             g = static_cast<float>(rand()) / static_cast<float>(RAND_MAX);
219             b = static_cast<float>(rand()) / static_cast<float>(RAND_MAX);
220             tiempoAnterior = tiempoActual;
221         }
```

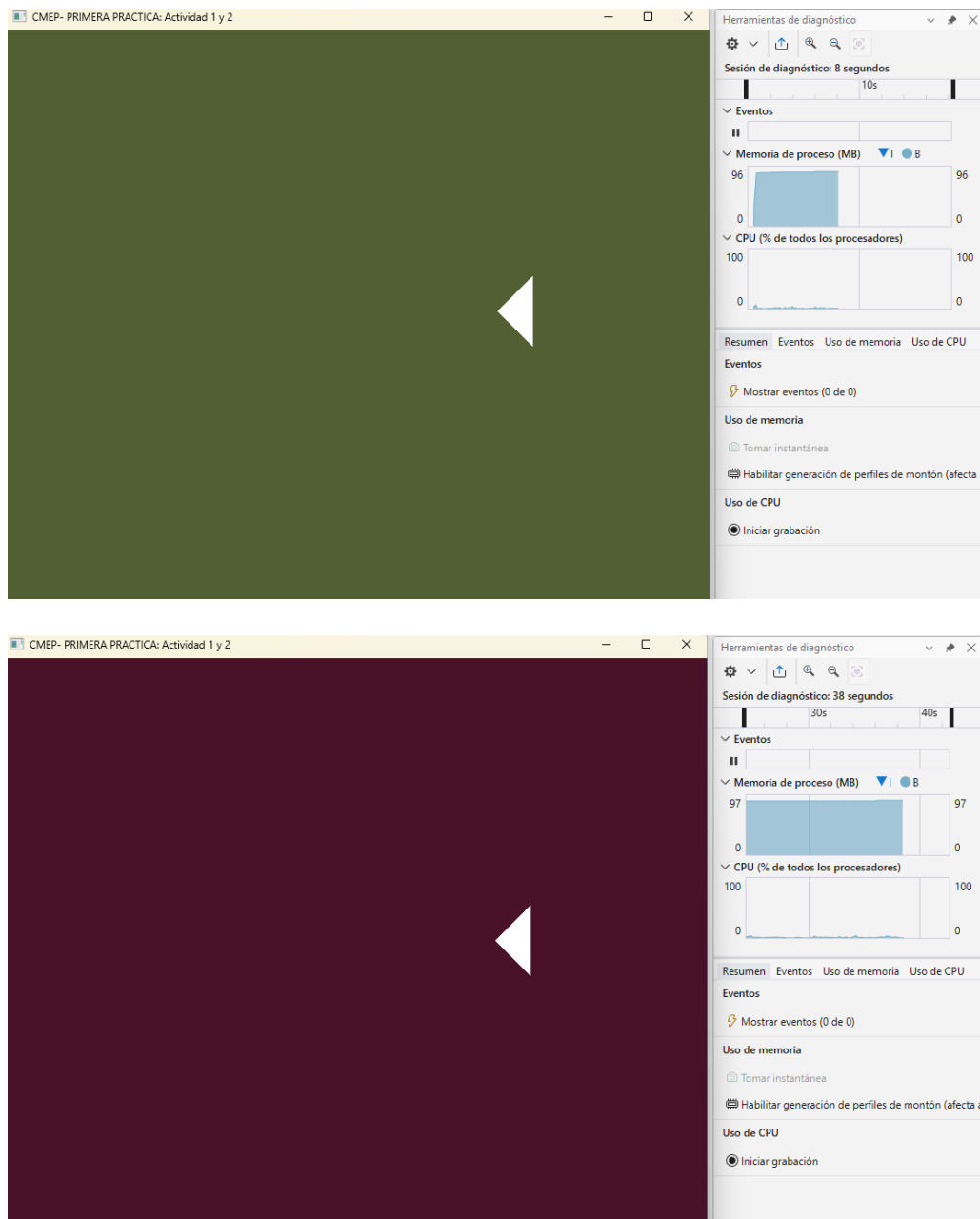
```

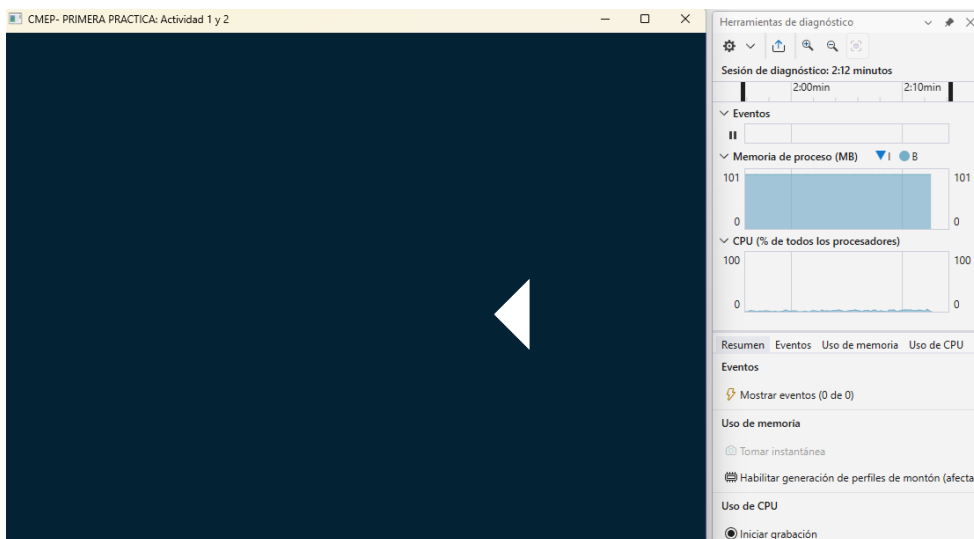
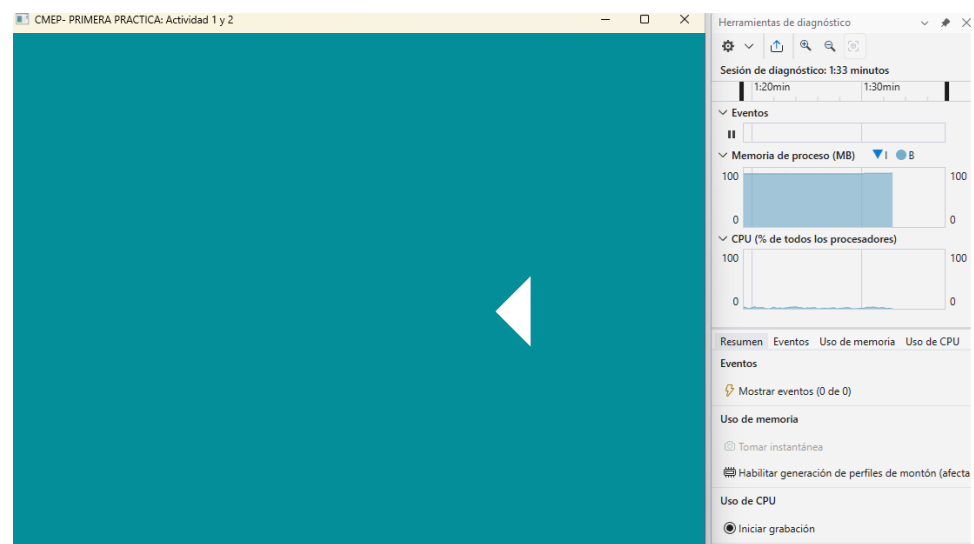
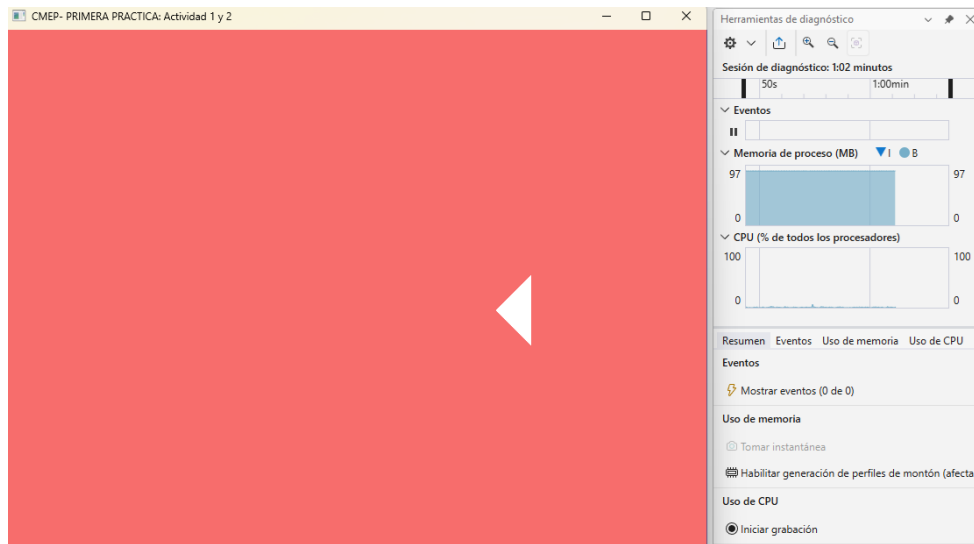
222 // Limpiar la ventana con el nuevo color aleatorio
223 glClearColor(r, g, b, 1.0f);
224 glClear(GL_COLOR_BUFFER_BIT);
225 // -----

```

EJECUCIÓN

Se comprobó que se generaron colores aleatorios cada dos segundos en cada ejecución. Se adjunta el cuadro de herramientas de diagnóstico para demostrar el cambio conforme pasan los segundos.





2.- Letras de mi nombre

Para la segunda actividad se pidió colocar en ventana las tres letras iniciales de nuestros nombres, en mi caso siendo las letras E, C y M, estas deberían ser creadas a partir de triángulos acomodados de formas diagonales de abajo hacia arriba.

Se comenzó dibujando en tableta las letras correspondientes para luego ir dividiéndolas en triángulos, obteniendo un total de 22, 8 para la letra E, 6 para la letra C y 8 más para la letra M. Se reutilizó la función de `CrearTriangulo` que nos ayudaba a dibujar un rombo y un trapecio en los ejercicios de clase y finalmente se obtuvieron las coordenadas para los 22 triángulos.

```
36 // CAMBIO 4: -----
37 void CrearTriangulo()
38 {
39     // Como estamos en 2D el ultimo valor (Z)
40     // lo dejamos siempre en 0 ---- X,Y,Z
41     GLfloat vertices[] = {
42
43         // LETRA E (Conformada por 8 triángulos):
44         // Rectangulo vertical
45         -0.6f, 0.3f, 0.0f, -0.6f, -0.2f, 0.0f, -0.5f, -0.2f, 0.0f, //Triangulo 1
46         -0.6f, 0.3f, 0.0f, -0.5f, -0.2f, 0.0f, -0.5f, 0.3f, 0.0f, //Triangulo 2
47         // Rectangulo superior
48         -0.5f, 0.3f, 0.0f, -0.5f, 0.2f, 0.0f, -0.3f, 0.2f, 0.0f, //Triangulo 3
49         -0.5f, 0.3f, 0.0f, -0.3f, 0.2f, 0.0f, -0.3f, 0.3f, 0.0f, //Triangulo 4
50         // Rectangulo medio
51         -0.5f, 0.1f, 0.0f, -0.5f, 0.0f, 0.0f, -0.4f, 0.0f, 0.0f, //Triangulo 5
52         -0.5f, 0.1f, 0.0f, -0.4f, 0.0f, 0.0f, -0.4f, 0.1f, 0.0f, //Triangulo 6
53         // Rectangulo inferior
54         -0.5f, -0.1f, 0.0f, -0.5f, -0.2f, 0.0f, -0.3f, -0.2f, 0.0f, //Triangulo 7
55         -0.5f, -0.1f, 0.0f, -0.3f, -0.2f, 0.0f, -0.3f, -0.1f, 0.0f, //Triangulo 8
56
57         // LETRA C (Conformada por 6 triángulos):
58         // Rectangulo vertical
59         -0.2f, 0.3f, 0.0f, -0.2f, -0.2f, 0.0f, -0.1f, -0.2f, 0.0f, //Triangulo 1
60         -0.2f, 0.3f, 0.0f, -0.1f, -0.2f, 0.0f, -0.1f, 0.3f, 0.0f, //Triangulo 2
61         // Rectangulo superior
62         -0.1f, 0.3f, 0.0f, -0.1f, 0.2f, 0.0f, 0.1f, 0.2f, 0.0f, //Triangulo 3
63         -0.1f, 0.3f, 0.0f, 0.1f, 0.2f, 0.0f, 0.1f, 0.3f, 0.0f, //Triangulo 4
64         // Rectangulo inferior
65         -0.1f, -0.1f, 0.0f, -0.1f, -0.2f, 0.0f, 0.1f, -0.2f, 0.0f, //Triangulo 5
66         -0.1f, -0.1f, 0.0f, 0.1f, -0.2f, 0.0f, 0.1f, -0.1f, 0.0f, //Triangulo 6
67
68         // LETRA M (Conformada por 8 triángulos):
69         //Rectangulo vertical izquierdo
70         0.2f, 0.3f, 0.0f, 0.2f, -0.2f, 0.0f, 0.3f, -0.2f, 0.0f, //Triangulo 1
71         0.2f, 0.3f, 0.0f, 0.3f, -0.2f, 0.0f, 0.3f, 0.3f, 0.0f, //Triangulo 2
72         //Rectangulo vertical derecho
73         0.5f, 0.3f, 0.0f, 0.5f, -0.2f, 0.0f, 0.6f, -0.2f, 0.0f, //Triangulo 3
74         0.5f, 0.3f, 0.0f, 0.6f, -0.2f, 0.0f, 0.6f, 0.3f, 0.0f, //Triangulo 4
75         // Diagonales
76         0.3f, 0.3f, 0.0f, 0.4f, 0.2f, 0.0f, 0.3f, 0.1f, 0.0f, //Triangulo 5
77         0.3f, 0.1f, 0.0f, 0.4f, 0.2f, 0.0f, 0.4f, 0.0f, 0.0f, //Triangulo 6
78         0.5f, 0.3f, 0.0f, 0.4f, 0.2f, 0.0f, 0.5f, 0.1f, 0.0f, //Triangulo 7
79         0.5f, 0.1f, 0.0f, 0.4f, 0.2f, 0.0f, 0.4f, 0.0f, 0.0f, //Triangulo 8
80     };
81 }
```

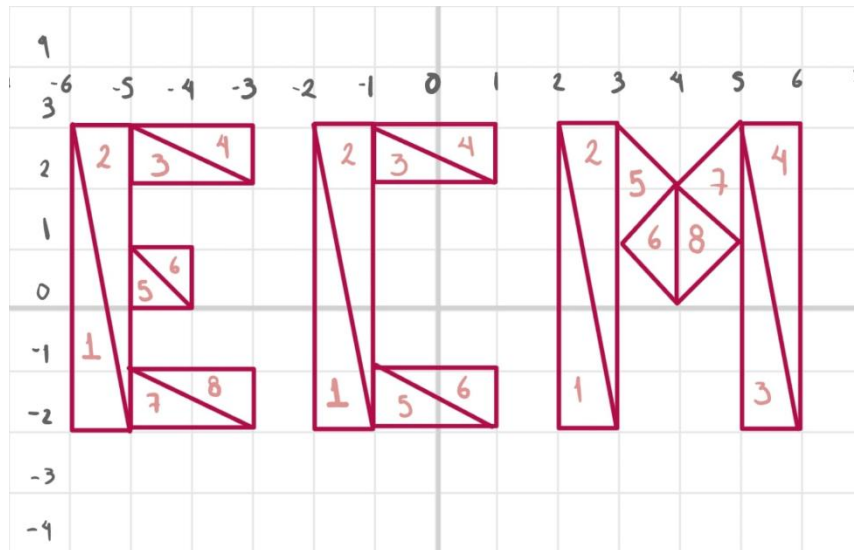
EJECUCIÓN

Se adjuntan letras semi completas y completas para verificar los triángulos.



PROBLEMAS PRESENTADOS

Para esta primera práctica no se presentaron problemas en cuanto a código, ya que las funciones de tiempo y aleatoriedad ya se habían trabajado anteriormente y bastó con revisar un poco la documentación de nuevo, únicamente se presentó un error de logística al indicar las coordenadas de los vértices para formar los triángulos, ya que quedaban triángulos extraños y sin conectarse correctamente. Al final se optó por usar un plano cartesiano en la tableta e ir tomando las referencias de coordenadas desde ahí para ya no hacerlo a ojo.



CONCLUSIONES

Sobre los ejercicios del reporte:

La complejidad de los ejercicios de momento fue baja, solo se tuvo que hacer algunos repastos sobre las librerías de tiempo y rand, creo que inclusive estas actividades fueron más sencillas que los ejercicios dejados en clase, aunque claramente se elevó la complejidad a diferencia de los anteriores, este cambio radica en que ya se hizo un buen análisis previo de los colores RGB en el código que se nos había proporcionado y que en el ejercicio de clase ya se había puesto un cambio cada 1.5 segundos.

Comentarios generales:

Las explicaciones del laboratorio fueron claras, pero nunca está demás detallar por escrito las actividades en la plataforma, inclusive poner ejemplos o pedir algún enlace al vídeo de ejecución de nuestro código nunca estaría demás, en especial en este tipo de actividades donde se aprecia un cambio constante en alguna parte de la ejecución.

Cierre:

En esta primera práctica, la introducción a OpenGL fue clara, aunque soy consciente que conforme avance el curso surgirán más dudas y también más ideas, de momento el dominio parcial del espacio de coordenadas de la ventana quedó bien entendido, así como la comprensión del uso de colores dinámicos.

BIBLIGORAFÍA

Cplusplus.com. (s.f.). rand - C++ Reference. Recuperado el 18 de febrero de 2026, de <https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/rand>

Cppreference.com. (2023, 10 de octubre). std::time - cppreference.com. Recuperado el 18 de febrero de 2026, de <https://en.cppreference.com/w/cpp/chrono/c/time>

GLFW. (s.f.). Input guide: Time input. Recuperado el 18 de febrero de 2026, de https://www.glfw.org/docs/latest/input_guide.html#time